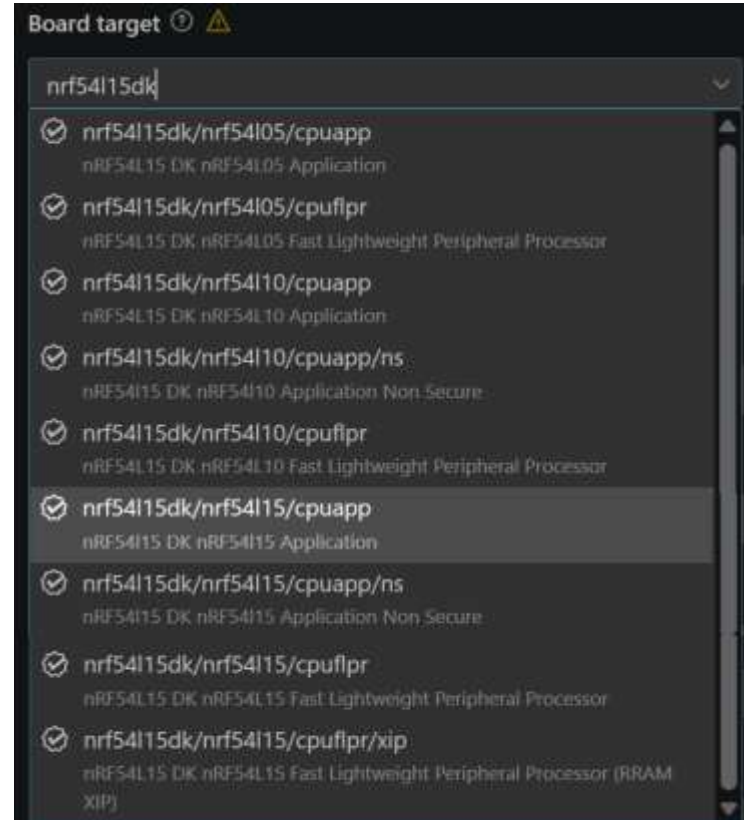
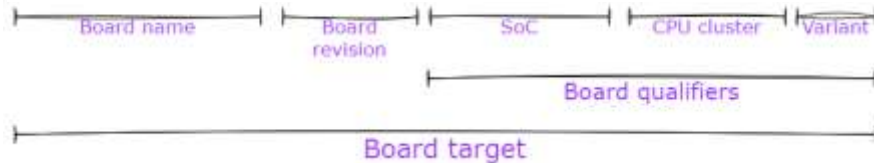


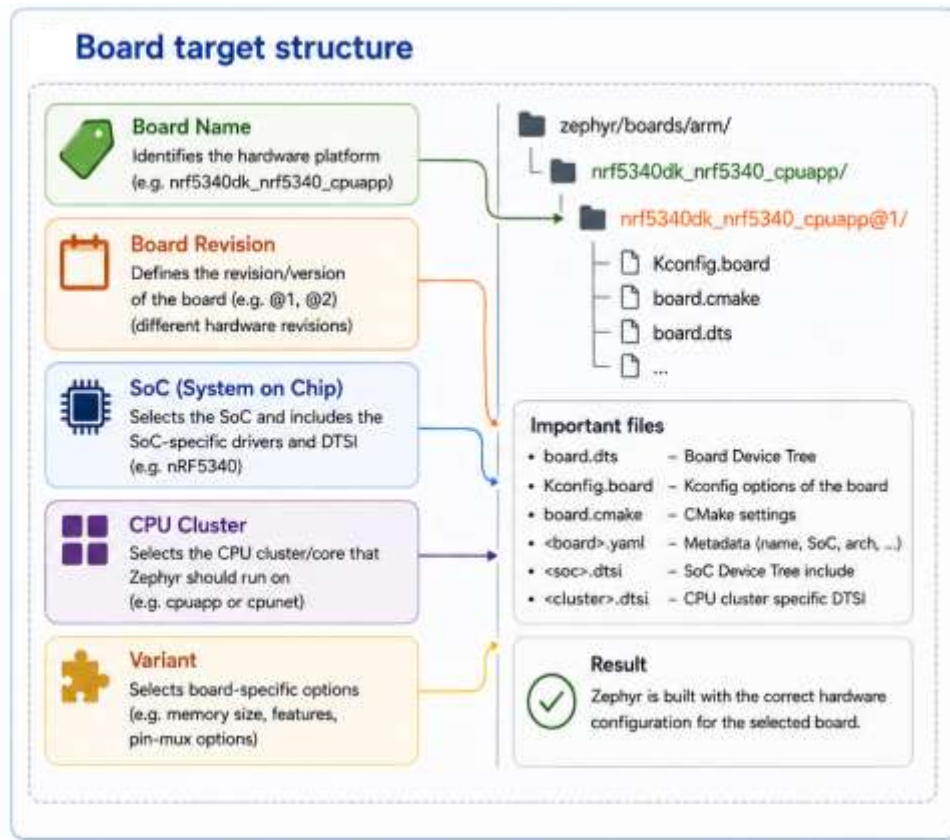
Board Target and Custom Boards

Board Terminology

nrf54L15dk @ 1.0.0 /nrf54L15 / cpuapp / ns



Board Target Structure



How to Create a Custom Board Target in Zephyr RTOS

A board target tells Zephyr everything about your hardware so it can build the correct firmware.

1 Choose a location for your board

Place your board definition inside your application repository (or a module) under `boards/<arch>/<vendor>/<board>/`.



Tip

Keeping the board in your app or module allows you to share it easily and upstream it later.

2 Define the board

Create the mandatory files that describe your board to Zephyr and Kconfig.

File	Purpose
Kconfig.board	Provides the Kconfig symbol for the board (name, prompt, dependencies).
board.cmake	Sets board-specific CMake variables (sources, include dirs, runner, etc.).
board.dts	Describes the hardware using Devicetree (SoC, peripherals, pins, LEDs, buttons, ...).

3 Add Kconfig symbol

Define a unique Kconfig symbol for your board.

`boards/arm/myvendor/myboard/Kconfig.board`

```
config BOARD_MYVENDOR_MYBOARD
    bool "MyVendor MyBoard"
    depends on SOC_FOO_BAR # Your SoC
    select HAS_DTS
    help
        Enable support for MyVendor MyBoard.
```



Result:

Enables the board to be selected with `-b myboard` (or `-p auto`).

4 Describe the hardware with Devicetree

`board.dts` tells Zephyr about your SoC, peripherals, pinctrl, LEDs, buttons, aliases, chosen settings, etc.

```
boards/arm/myvendor/myboard/board.dts
/dts-v1/;

#include <soc/foo_bar.dtsi>

/ {
    model = "MyVendor MyBoard";
    compatible = "myvendor,myboard", "vendor,foo-bar-soc";

    chosen {
        zephyr,console = &uart0;
        zephyr,shell-uart = &uart0;
    };

    aliases {
        led0 = &led_green;
        sw0 = &button0;
    };
};
```



Devicetree includes

- SoC .dtsi
- Pinmux
- Peripherals
- LEDs, Buttons
- Aliases
- Chosen
- Regulators
- Clocks
- ...

5 Provide build information with CMake

`board.cmake` tells Zephyr how to build for your board.

```
boards/arm/myvendor/myboard/board.cmake

# Add board specific sources
set(BOARD_SOURCES board.c)

# Add include directories
zephyr_include_directories(${CMAKE_CURRENT_LIST_DIR})

# (Optional) Set runner arguments
set(BOARD_RUNNER_ARGS --device /dev/ttyACM0)

# (Optional) Flash and debug configuration
include(${ZEPHYR_BASE}/boards/common/board.cmake)
```



Common uses

- Add board initialization sources (board.c)
- Include directories
- Set runner, flash and debug options
- Add extra CMake logic if needed

6 Build and use your board

Once the files are in place, Zephyr can build your application for your board.

```
west build -b myboard path/to/my_app
```



SoC & Peripherals
(Devicetree)



Board Init & Build
(CMake)



Kconfig Options
(Board)



Firmware for
Your Board

Best practices

- Reuse existing SoC .dtsi and bindings.
- Keep board specific things in the board folder.
- Use meaningful names and follow Zephyr coding style.
- Test with `west build -b myboard samples/hello_world`
- Consider upstreaming your board to Zephyr!

In short

1

Choose location
`boards/arch/vendor/board/`

2

Create files
`Kconfig.board, board.cmake, board.dts`

3

Describe hardware
Devicetree for your board

4

Add build info
CMake for sources, runners, options

5

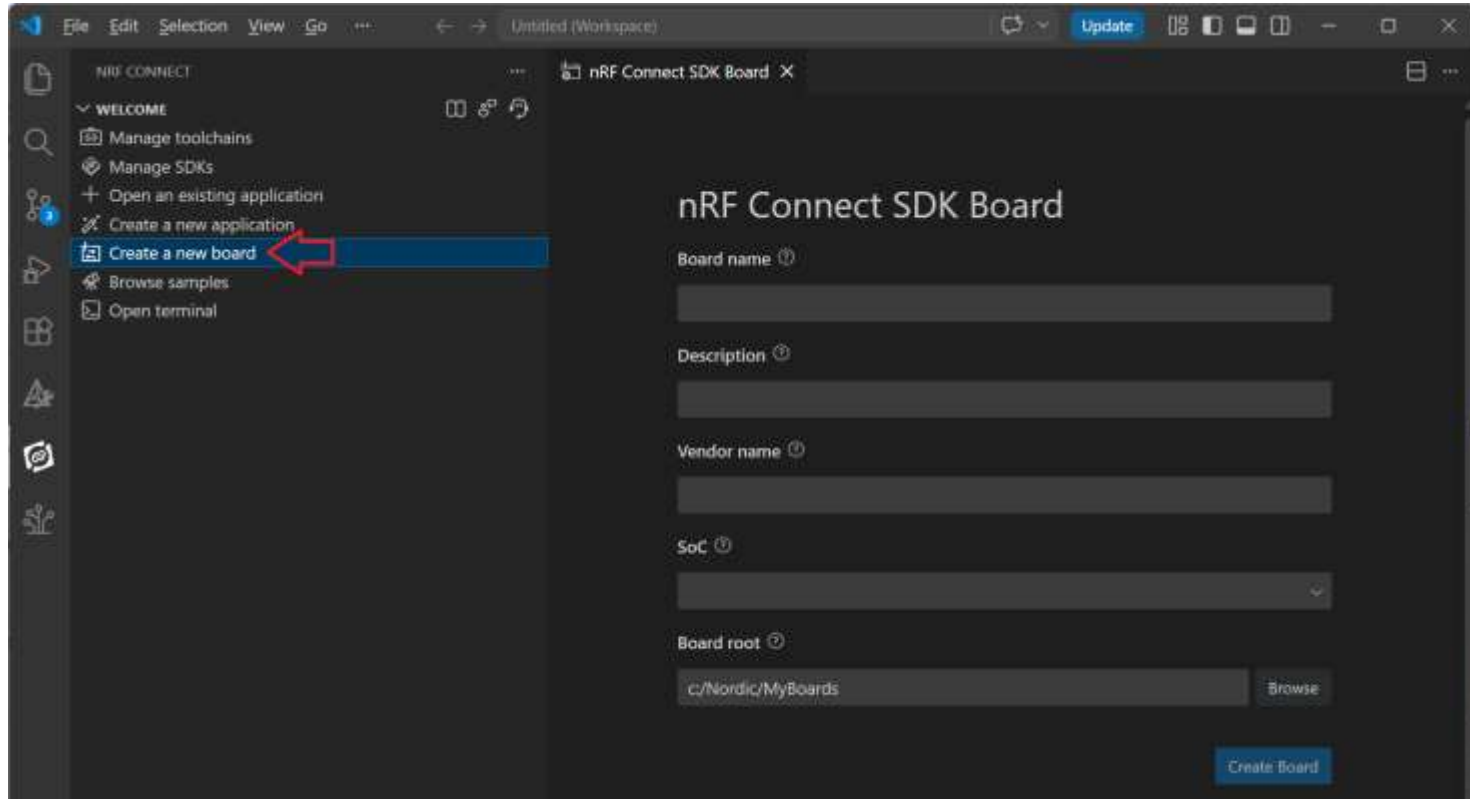
Build it
`west build -b myboard`

Done!



Your custom board target
is ready to use

nRF Connect: Create a new board



Example: Create a new Board

nRF Connect SDK Board

Board name ⓘ
BoardChris

Description ⓘ
This is the board for Chris' project

Vendor name ⓘ
chris

SoC ⓘ
nRF5340-QKAA

Board root ⓘ
c:/Nordic/MyBoards Browse

Create Board

OS (C:) > Nordic > MyBoards > boards > chris > BoardChris

Name	Date modified	Type	Size
board.cmake	5/15/2026 7:22 AM	CMake Source File	1 KB
board.yml	5/15/2026 7:22 AM	Yaml Source File	1 KB
BoardChris_nrf5340_cpuapp.dts	5/15/2026 7:22 AM	DTS File	1 KB
BoardChris_nrf5340_cpuapp.yaml	5/15/2026 7:22 AM	Yaml Source File	1 KB
BoardChris_nrf5340_cpuapp_defconfig	5/15/2026 7:22 AM	File	1 KB
BoardChris_nrf5340_cpuapp_ns.dts	5/15/2026 7:22 AM	DTS File	1 KB
BoardChris_nrf5340_cpuapp_ns.yaml	5/15/2026 7:22 AM	Yaml Source File	1 KB
BoardChris_nrf5340_cpuapp_ns_defconfig	5/15/2026 7:22 AM	File	1 KB
BoardChris_nrf5340_cpunet.dts	5/15/2026 7:22 AM	DTS File	1 KB
BoardChris_nrf5340_cpunet.yaml	5/15/2026 7:22 AM	Yaml Source File	1 KB
BoardChris_nrf5340_cpunet_defconfig	5/15/2026 7:22 AM	File	1 KB
BoardChris_cpuapp_partitioning.dtsi	5/15/2026 7:22 AM	DTSI File	4 KB
BoardChris-pinctrl.dtsi	5/15/2026 7:22 AM	DTSI File	1 KB
BoardChris-shared_sram.dtsi	5/15/2026 7:22 AM	DTSI File	1 KB
Kconfig.BoardChris	5/15/2026 7:22 AM	BOARDCHRIS File	1 KB
Kconfig.defconfig	5/15/2026 7:22 AM	DEFCONFIG File	2 KB
Kconfig.sysbuild	5/15/2026 7:22 AM	SYSBUILD File	1 KB
pre_dt_board.cmake	5/15/2026 7:22 AM	CMake Source File	1 KB

Example: Using created Board

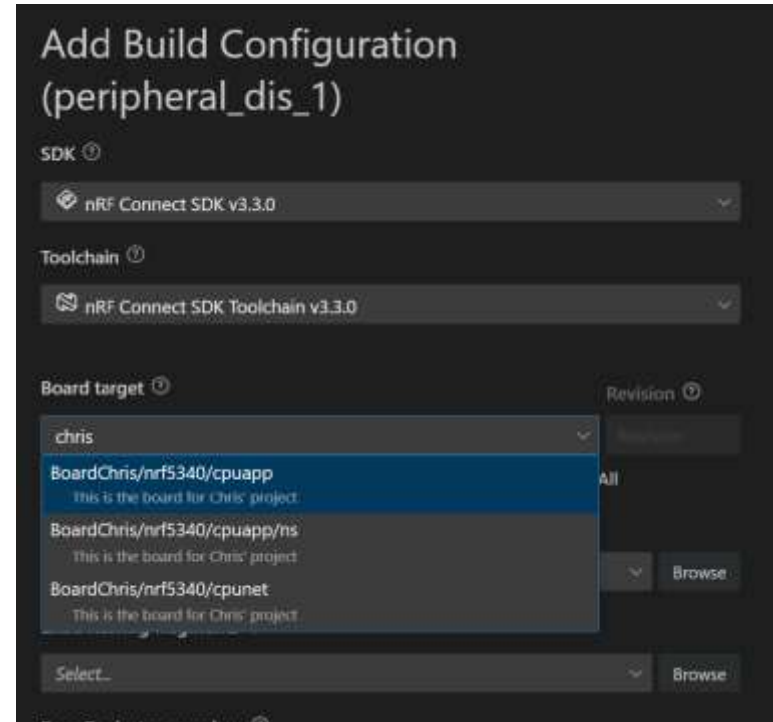
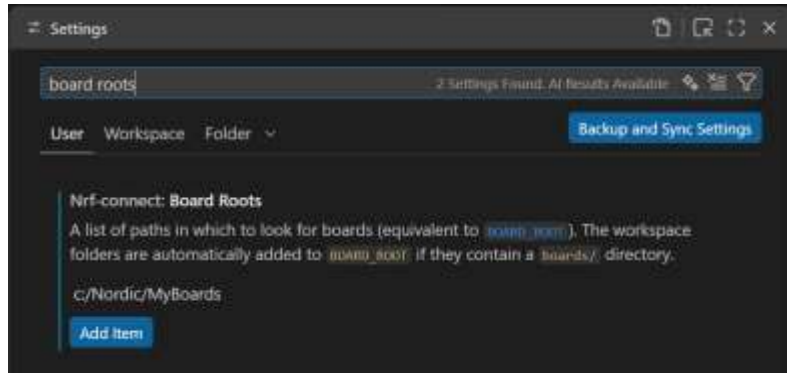
Command Line: Let Zephyr know where the board is:

CMakeLists.txt

```
list(APPEND BOARD_ROOT ${CMAKE_CURRENT_SOURCE_DIR})  
  
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})  
  
project(my_app)
```

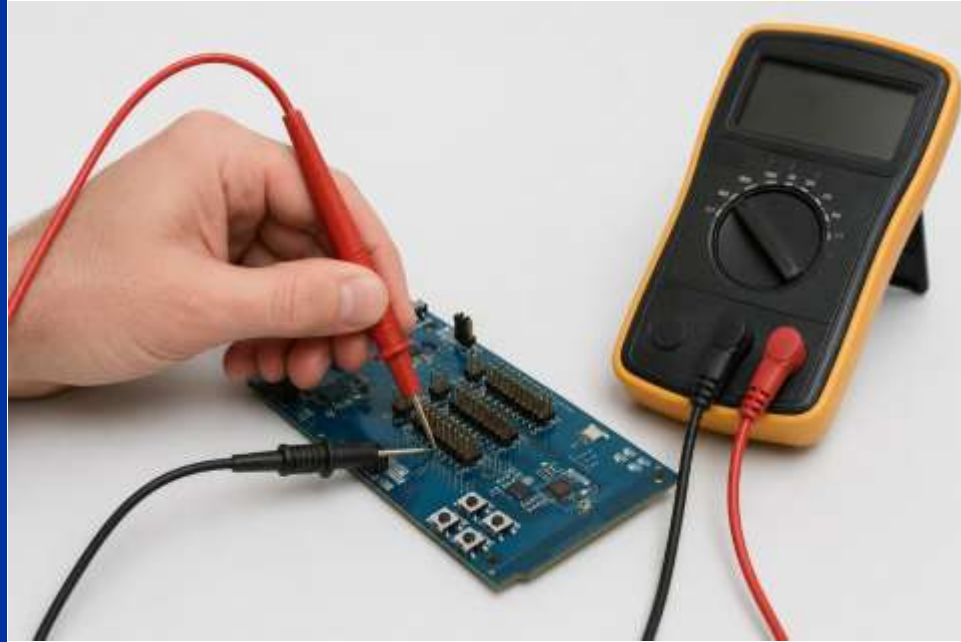
OR

VS Code: File | Preferences | Settings:



Hands-on

- Create a custom Board





Step 1 - Explore the Existing Board

Before creating a custom board, it is important to understand the files that define an existing board. The nRF54L15 DK board contains all required configuration files that will be reused as a template.

1. Locate the Zephyr board directory.
2. Navigate to: **zephyr/boards/nordic/nrf54l15dk**
3. Inspect the available files.
4. Identify:
 - board.yml
 - Kconfig.board
 - Kconfig.defconfig
 - *.dts
 - *.yaml
 - board.cmake



Step 2 - Using *Create a new board* Wizard

5. Click on *Create a new board* button within the **nRF Connect** extension.
6. Fill out the form in the Board Wizard window. Use the following settings:
 - Board name: `lab_board`
 - Description: `my own custom board`
 - Vendor name: `nordic`
 - SoC: `nRF54L15-QFAA`
 - Board root: `c:/Nordic/myBoards`



Tip: The board root location setting does not automatically include in most cases the board to the board list. We will do this in the next step. There are different possibilities:

- Application-local / freestanding folder: Inside the application source tree or in a boards folder in application source tree.
- Freestanding folder elsewhere: Including path via CMakeLists.txt or VS Code Board root setting.
- west workspace: for example, by using Application Example Add-on
- In-tree (inside the Zephyr repository): only if Zephyr upstream merges are planned



Step 3 - Modify the DeviceTree

7. Open the board DTS file: **lab_board_nrf54l15_cpuapp.dts**
8. Review the existing peripheral configuration.
9. Add the GPIO assignments for your LED. We will use nRF54L15DK's LED3, which is on GPIO P1.14.

```
/ {  
    leds {  
        compatible = "gpio-leds";  
  
        myled: myled {  
            gpios = <&gpio1 14 GPIO_ACTIVE_HIGH>;  
            label = "My LED";  
        };  
    };  
};
```



Step 4 - Modify KCONFIG

10. Add GPIO software module by adding following line in file **lab_board_nrf54l15_cpuapp_defconfig**.

```
CONFIG_GPIO=y
```



Step 5 - Add custom board folder to Board Root list

11. Add the path of our custom board to the board root list within Visual Studio Code:

File -> Preferences -> Settings -> Extensions -> nRF Connect -> Board Roots

Add your custom board path there, e.g. **c:/Nordic/myBoards**



Step 6 - Use custom Board in *blinky* sample

12. Copy Zephyr's **blinky** sample. ([zephyr/samples/basic/blinky](#))
13. **Add build configuration** and use following settings:
Board Target: lab_board/nrf54l15/cpuapp
14. Click on the **Generate and Build** button.

Step 7 - Flash and Test

15. Download the project by clicking on **Flash** button.
16. Does the LED blink?